



آزمایشگاه طراحی سیستم‌های دیجیتال

ترم تابستان ۱۴۰۵

استاد: دکتر انصاری

آزمایش هفتم

طراحی UART

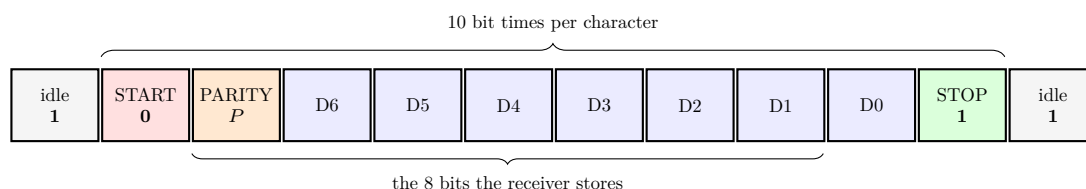
خلاصه

در این آزمایش یک Universal Asynchronous Receiver Transmitter طراحی و پیاده‌سازی شده است. طبق صورت آزمایش، فرستنده هر بار یک کد ASCII هفت‌بیتی را به‌صورت سریال بیرون می‌فرستد و گیرنده پس از دریافت بیت شروع، هشت بیت بعدی — یعنی توازن و داده — را در یک ثبات ۸ بیتی ذخیره می‌کند. در پایان دو واحد کامل به یکدیگر وصل شده‌اند تا نشان داده شود مبادله در هر دو جهت درست انجام می‌شود.

نکته‌ای که تمام طراحی حول آن می‌چرخد این است که UART یک پروتکل ناهم‌گام است: هیچ سیم کلاکی بین دو طرف کشیده نمی‌شود. تنها چیزی که فرستنده و گیرنده در آن توافق دارند نرخ بیت و قالب فریم است. بنابراین گیرنده باید خودش لحظه‌ی شروع فریم را از روی خط تشخیص دهد و سپس بدون کمک گرفتن از کلاک فرستنده، محل درست نمونه‌برداری هر بیت را حدس بزند.

قالب فریم

صورت آزمایش قالب را دقیقاً مشخص کرده است: یک بیت شروع، سپس یک بیت توازن، سپس ۷ بیت داده و در انتها حداقل یک بیت خاتمه — در مجموع ۱۰ بیت.



دو تفاوت با UART استاندارد وجود دارد و هر دو مستقیماً از صورت آزمایش می‌آید:

۱. بیت توازن قبل از داده فرستاده می‌شود، نه بعد از آن. پیامد این انتخاب برای سخت‌افزار مهم است: فرستنده باید تمام کاراکتر را پیش از شروع ارسال در اختیار داشته باشد، چون توازن تابعی از همه‌ی هفت بیت است. به همین دلیل داده به‌صورت موازی بارگذاری می‌شود و توازن در همان لحظه‌ی بارگذاری به‌صورت ترکیبی محاسبه می‌شود.

۲. بار مفید ۷ بیت است و گیرنده ۸ بیت پس از بیت شروع را در یک ثبات ذخیره می‌کند.

چرا داده MSB اول فرستاده می‌شود

در UART استاندارد داده از کم‌ارزش‌ترین بیت شروع می‌شود. اینجا عمداً برعکس عمل شده است و دلیلش خواسته‌ی خود صورت آزمایش است: گیرنده باید ۸ بیت را در یک ثبات ۸ بیتی بریزد.

گیرنده یک ثبات لغزشی است که هر بیت جدید را از سمت کم‌ارزش وارد می‌کند. اگر ترتیب ارسال $P, D_6, D_5, \dots$ باشد، محتوای ثبات پس از هشت شیفต์ دقیقاً این است:

$$rx_reg = \{P, D6, D5, D4, D3, D2, D1, D0\}$$

یعنی بیت توازن در پرارزش‌ترین جایگاه و کد ASCII بدون هیچ جابه‌جایی در هفت بیت پایین. اگر داده LSB اول فرستاده می‌شد، همان ثبات کد ASCII را برعکس نگه می‌داشت و برای استفاده از آن باید یک معکوس‌کننده‌ی بیتی اضافه می‌شد — منطقی که هیچ کاری جز جبران یک انتخاب بد نمی‌کند.

تولید نرخ بیت و نمونه‌برداری ۱۶ برابر

هر واحد یک شمارنده‌ی تقسیم دارد که پالسی به نام `os_tick` با نرخ ۱۶ برابر نرخ بیت تولید می‌کند. فرستنده و گیرنده‌ی یک واحد هر دو از همین یک پالس استفاده می‌کنند، پس هزینه‌ی هر UART یک شمارنده است نه دو تا.

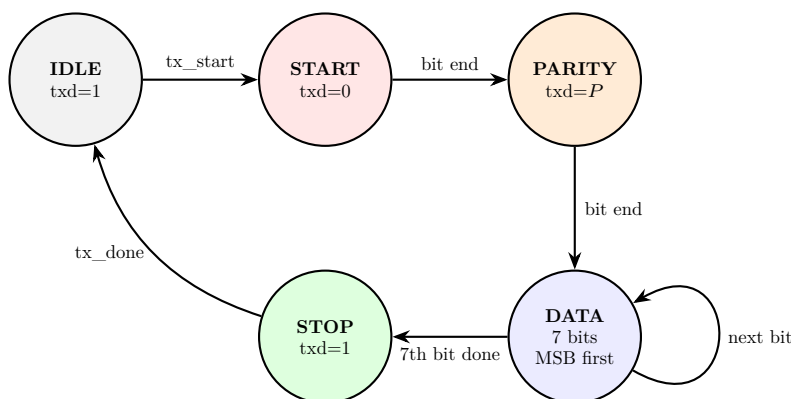
```
DIVISOR = f_clk / (16 * baud) // synthesis: e.g. 50 MHz / (16 * 115200) = 27
```

نسبت تقسیم پارامتر است تا در سنتز مقدار واقعی و در شبیه‌سازی مقداری کوچک استفاده شود؛ در غیر این صورت هر تست‌بنچ باید میلیون‌ها کلاک بی‌کار را شبیه‌سازی می‌کرد.

اما چرا اصلاً ۱۶ برابر؟ چون گیرنده کلاک فرستنده را ندارد. تنها لنگرگاه زمانی‌اش لبه‌ی پایین‌رونده‌ی بیت شروع است و این لبه با دقت یک پالس نمونه‌برداری تشخیص داده می‌شود. با ۱۶ برابر، این خطای اولیه $\frac{1}{16}$ یک بیت است و نقطه‌ی نمونه‌برداری آن قدر به وسط بیت نزدیک می‌ماند که تا انتهای فریم هم داخل بیت درست باقی بماند. جزئیات این حاشیه‌ی خطا در بخش تست‌ها می‌آید.

فرستنده

فرستنده یک ماشین حالت ساده است که هر حالت آن دقیقاً ۱۶ پالس `os_tick` طول می‌کشد:



دو نکته‌ی پیاده‌سازی:

بیت شروع در همان لبه‌ی بارگذاری روی خط می‌رود. اگر منتظر اولین `os_tick` می‌ماندیم، اولین بیت به اندازه‌ی فاز باقی‌مانده‌ی شمارنده کش می‌آمد. با پایین بردن فوری خط و صفر کردن هم‌زمان شمارنده‌ی داخل بیت، طول بیت شروع دقیقاً ۱۶ پالس می‌شود و همه‌ی بیت‌های بعدی هم به همان اندازه.

درخواست جدید در میانه‌ی فریم نادیده گرفته می‌شود، نه صف. سیگنال `tx_busy` برای همین بیرون داده شده است. اگر درخواست‌ها صف می‌شدند، به یک بافر نیاز بود؛ و اگر فریم جاری را قطع می‌کردند، گیرنده‌ی طرف مقابل یک فریم ناقص می‌دید. این رفتار در تست‌بنچ صریحاً بررسی شده است.

```
1 `include "uart_pkg.vh"
2
3 // UART transmitter.
```

```

4 //
5 // Sends one 10-bit frame per request: START(0) PARITY D6..D0 STOP(1).
6 // Each bit lasts `UART_OVERSAMPLE` pulses of os_tick, so the module needs no
7 // divider of its own -- it shares the one inside the enclosing uart unit.
8 //
9 // ODD_PARITY = 0 -> even parity (parity bit makes the count of ones even)
10 // ODD_PARITY = 1 -> odd parity
11 module uart_tx #(
12     parameter ODD_PARITY = 1'b0
13 ) (
14     input wire          Clk,
15     input wire          RstN,
16     input wire          os_tick,      // 16x baud pulse
17     input wire          tx_start,     // load & send (ignored while busy)
18     input wire [ `UART_DATA_BITS-1:0] tx_data, // 7-bit ASCII code
19     output reg          txd,          // serial line, idles high
20     output wire         tx_busy,
21     output reg          tx_done       // 1 clk pulse at end of stop bit
22 );
23     localparam [2:0] IDLE   = 3'd0,
24                     START  = 3'd1,
25                     PARITY  = 3'd2,
26                     DATA   = 3'd3,
27                     STOP    = 3'd4;
28
29     reg [2:0] st;
30     reg [3:0] os;      // 0..15 within the current bit
31     reg [2:0] idx;     // which data bit is on the wire
32     reg [ `UART_DATA_BITS-1:0] sh; // shifts out MSB first
33     reg par;
34
35     // The character is known in full at load time, so the parity bit can be
36     // produced combinationaly and sent ahead of the data.
37     wire par_of_data = ODD_PARITY ? ~(^tx_data) : (^tx_data);
38
39     // A bit time is over on the last oversampling pulse of that bit.
40     wire bit_end = os_tick && (os == `UART_OVERSAMPLE - 1);
41
42     assign tx_busy = (st != IDLE);
43
44     always @(posedge Clk or negedge RstN) begin
45         if (!RstN) begin
46             st      <= IDLE;
47             os      <= 4'd0;
48             idx      <= 3'd0;
49             sh       <= { `UART_DATA_BITS{1'b0}};
50             par      <= 1'b0;
51             txd      <= 1'b1;
52             tx_done  <= 1'b0;
53         end else begin
54             tx_done <= 1'b0;
55
56             if (st != IDLE && os_tick)
57                 os <= bit_end ? 4'd0 : os + 4'd1;
58
59             case (st)
60                 IDLE: begin
61                     txd <= 1'b1;
62                     if (tx_start) begin
63                         sh <= tx_data;
64                         par <= par_of_data;
65                         txd <= 1'b0;          // start bit goes out immediately
66                         os  <= 4'd0;
67                         idx <= `UART_DATA_BITS - 1;
68                         st  <= START;
69                     end
70                 end
71
72                 START: if (bit_end) begin
73                     txd <= par;
74                     st  <= PARITY;
75                 end
76
77                 PARITY: if (bit_end) begin
78                     txd <= sh[ `UART_DATA_BITS-1];
79                     sh  <= {sh[ `UART_DATA_BITS-2:0], 1'b0};
80                     st  <= DATA;
81                 end
82
83                 DATA: if (bit_end) begin
84                     if (idx == 3'd0) begin
85                         txd <= 1'b1;          // stop bit
86                         st  <= STOP;
87                     end else begin
88                         txd <= sh[ `UART_DATA_BITS-1];
89                         sh  <= {sh[ `UART_DATA_BITS-2:0], 1'b0};
90                         idx <= idx - 3'd1;

```

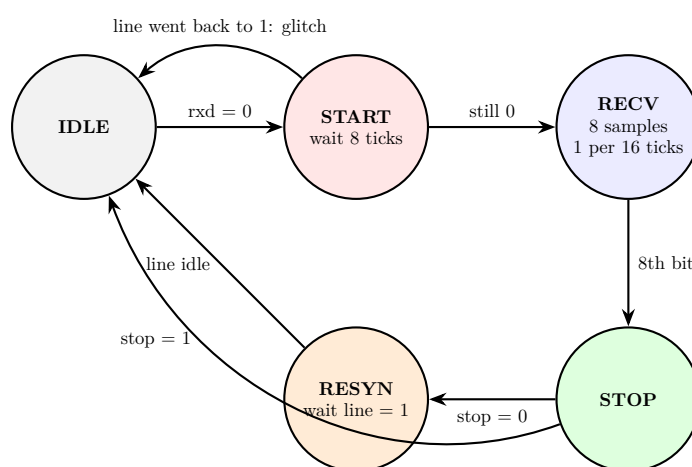
```

91         end
92     end
93
94     STOP: if (bit_end) begin
95         txd     <= 1'b1;           // line stays idle high
96         tx_done <= 1'b1;
97         st      <= IDLE;
98     end
99
100     default: st <= IDLE;
101 endcase
102 end
103 endmodule
104

```

گیرنده

گیرنده کار سختی دارد: باید بدون کلاک مشترک بفهمد هر بیت کجا شروع و کجا تمام می‌شود.



الف) هم‌گام‌سازی ورودی. خط سریال از دامنه‌ی زمانی دیگری (یا روی برد، از یک پایه) می‌آید و نباید مستقیم به ماشین حالت وصل شود. دو فلیپ‌فلاپ پشت سر هم احتمال metastability را عملاً از بین می‌برند.

ب) رد کردن نویز به جای شروع فریم. با پایین آمدن خط، گیرنده نیم بیت صبر می‌کند و دوباره خط را می‌بیند. اگر خط بالا رفته باشد، آنچه دیده شده یک سوزن نویزی بوده و گیرنده به IDLE برمی‌گردد. اگر هنوز پایین باشد، این واقعا بیت شروع است — و از قضا این لحظه دقیقاً وسط بیت شروع است.

ج) نمونه‌برداری از وسط هر بیت. از همان نقطه به بعد، هر ۱۶ پالس یک بار نمونه گرفته می‌شود. چون نقطه‌ی شروع وسط بیت شروع بود، همه‌ی نمونه‌های بعدی هم دقیقاً وسط بیت خودشان می‌افتند — بیشترین فاصله‌ی ممکن از هر دو لبه، یعنی بیشترین تحمل نسبت به اختلاف نرخ بیت دو طرف.

د) بازگشت به هم‌گامی پس از خطای فریم. اگر بیت خاتمه صفر خوانده شود، فریم خراب بوده و خط هنوز پایین است. برگشتن مستقیم به IDLE یعنی همان صفر بلافاصله به عنوان بیت شروع بعدی تفسیر شود و یک فریم زباله‌ی دیگر تولید کند. به همین دلیل حالت RESYN اضافه شده که تا بی‌کار شدن خط منتظر می‌ماند. این رفتار هم در تست‌بنچ بررسی شده است.

```

1  `include "uart_pkg.vh"
2
3  // UART receiver.
4  //
5  // Waits for the start bit, then shifts the next 8 line bits -- the parity bit
6  // followed by the 7 data bits -- into one 8-bit register, exactly as the lab
7  // sheet asks. Because the transmitter sends the data MSB first, the register
8  // ends up holding {parity, D6..D0}: the ASCII code is simply its low 7 bits.
9  //

```

```

10 // Sampling strategy
11 // -----
12 // os_tick runs at `UART_OVERSAMPLE` times the baud rate. On the falling edge of
13 // the line the receiver waits half a bit time and re-checks the line: a short
14 // glitch is rejected there instead of being mistaken for a character. From
15 // that point on it samples once every `UART_OVERSAMPLE` ticks, which keeps every
16 // sample in the middle of its bit.
17 module uart_rx #(
18     parameter ODD_PARITY = 1'b0
19 ) (
20     input wire Clk,
21     input wire RstN,
22     input wire os_tick,
23     input wire rxd,          // serial line
24
25     output reg [`UART_RX_REG_BITS-1:0] rx_reg,    // {parity, D6..D0}
26     output wire [`UART_DATA_BITS-1:0] rx_data,    // 7-bit ASCII code
27     output reg rx_valid,        // 1 clk pulse per frame
28     output reg parity_err,
29     output reg frame_err,
30     output wire rx_busy
31 );
32
33 localparam integer OS = `UART_OVERSAMPLE;
34
35 localparam [2:0] IDLE = 3'd0,
36                  START = 3'd1,
37                  RECV = 3'd2,
38                  STOP = 3'd3,
39                  RESYN = 3'd4;    // wait for the line to go idle again
40
41 // Two-flop synchroniser: rxd comes from another clock domain (or, on the
42 // board, from a pin) and must not be used directly in the FSM.
43 reg rx_s0, rx_s1;
44 always @(posedge Clk or negedge RstN) begin
45     if (!RstN) {rx_s0, rx_s1} <= 2'b11;
46     else {rx_s0, rx_s1} <= {rxd, rx_s0};
47 end
48
49 reg [2:0] st;
50 reg [3:0] os;
51 reg [2:0] nbit;
52 reg [`UART_RX_REG_BITS-1:0] sh;
53
54 assign rx_data = rx_reg[`UART_DATA_BITS-1:0];
55 assign rx_busy = (st != IDLE);
56
57 // Expected parity of the 7 data bits currently in the shift register.
58 wire exp_par = ODD_PARITY ? ~(^sh[`UART_DATA_BITS-1:0]) : (^sh[`UART_DATA_BITS-1:0]);
59
60 always @(posedge Clk or negedge RstN) begin
61     if (!RstN) begin
62         st <= IDLE;
63         os <= 4'd0;
64         nbit <= 3'd0;
65         sh <= {'UART_RX_REG_BITS{1'b0}};
66         rx_reg <= {'UART_RX_REG_BITS{1'b0}};
67         rx_valid <= 1'b0;
68         parity_err <= 1'b0;
69         frame_err <= 1'b0;
70     end else begin
71         rx_valid <= 1'b0;
72
73         case (st)
74             IDLE: if (!rx_s1) begin    // line went low -> possible start bit
75                 os <= 4'd0;
76                 st <= START;
77             end
78
79             // Half a bit time later, confirm it really is a start bit.
80             // The comparison is against OS/2-1 because `os` starts at 0
81             // and is tested BEFORE it is incremented: counting 0..OS/2-1
82             // is exactly OS/2 ticks. Using OS/2 here would spend an extra
83             // tick and push every later sample off the middle of its bit.
84             START: if (os_tick) begin
85                 if (os == OS/2 - 1) begin
86                     if (rx_s1) begin
87                         st <= IDLE;    // glitch, not a character
88                     end else begin
89                         os <= 4'd0;
90                         nbit <= 3'd0;
91                         st <= RECV;
92                     end
93                 end else begin
94                     os <= os + 4'd1;
95                 end
96             end
97         endcase
98     end
99 end

```

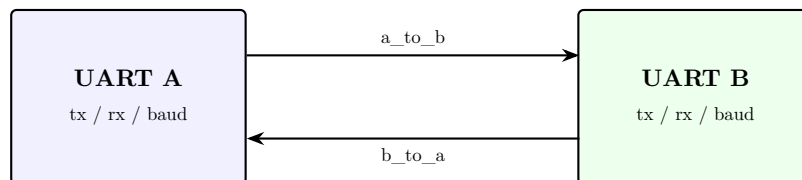
```

97 // One sample per bit, taken at the middle of the bit.
98 RECV: if (os_tick) begin
99   if (os == OS-1) begin
100     os   <= 4'd0;
101     sh   <= {sh[`UART_RX_REG_BITS-2:0], rx_s1};
102     nbit <= nbit + 3'd1;
103     if (nbit == `UART_RX_REG_BITS - 1) st <= STOP;
104   end else begin
105     os <= os + 4'd1;
106   end
107 end
108
109 STOP: if (os_tick) begin
110   if (os == OS-1) begin
111     os   <= 4'd0;
112     rx_reg <= sh;
113     parity_err <= (sh[`UART_RX_REG_BITS-1] != exp_par);
114     frame_err <= ~rx_s1; // stop bit must be high
115     rx_valid <= 1'b1;
116     st <= rx_s1 ? IDLE : RESYN;
117   end else begin
118     os <= os + 4'd1;
119   end
120 end
121
122 // A broken stop bit leaves the line low; starting a new frame
123 // from there would just produce more garbage, so wait until
124 // the line is idle again before re-arming.
125 RESYN: if (rx_s1) st <= IDLE;
126
127 default: st <= IDLE;
128 endcase
129 end
130 endmodule
131

```

اتصال دو واحد

خواسته‌ی پایانی صورت آزمایش این است که دو واحد به یکدیگر وصل شوند. ماژول uart_link دقیقاً همین است: خروجی فرستنده‌ی هر واحد به ورودی گیرنده‌ی واحد دیگر می‌رود و این دو سیم تمام «کابل» بین دو دستگاه‌اند.



no shared clock – only the agreed baud rate and frame format

دو واحد پارامتر تقسیم جداگانه دارند. با مقادیر برابر، لینک کاملاً هم‌نرخ است؛ با مقادیر کمی متفاوت، دو بردی مدل می‌شوند که کریستال‌هایشان با هم اختلاف دارند — و این دقیقاً همان چیزی است که نمونه‌برداری ۱۶ برابر باید از پس آن بربایند.

```

1 `include "uart_pkg.vh"
2
3 // Two UART units wired to each other -- the demonstration the lab sheet asks
4 // for. Unit A's transmit line is unit B's receive line and vice versa, so the
5 // two lines a_to_b / b_to_a are the whole "cable".
6 //
7 // The two units are given SEPARATE divisor parameters on purpose. With
8 // A_OS_DIVISOR == B_OS_DIVISOR the link is perfectly matched; making them
9 // slightly different models two boards whose crystals disagree, which is what
10 // the 16x oversampling in the receiver is there to survive.
11 module uart_link #(
12   parameter integer A_OS_DIVISOR = 16,
13   parameter integer B_OS_DIVISOR = 16,
14   parameter ODD_PARITY = 1'b0
15 ) (
16   input wire Clk,
17   input wire RstN,
18
19   // unit A

```

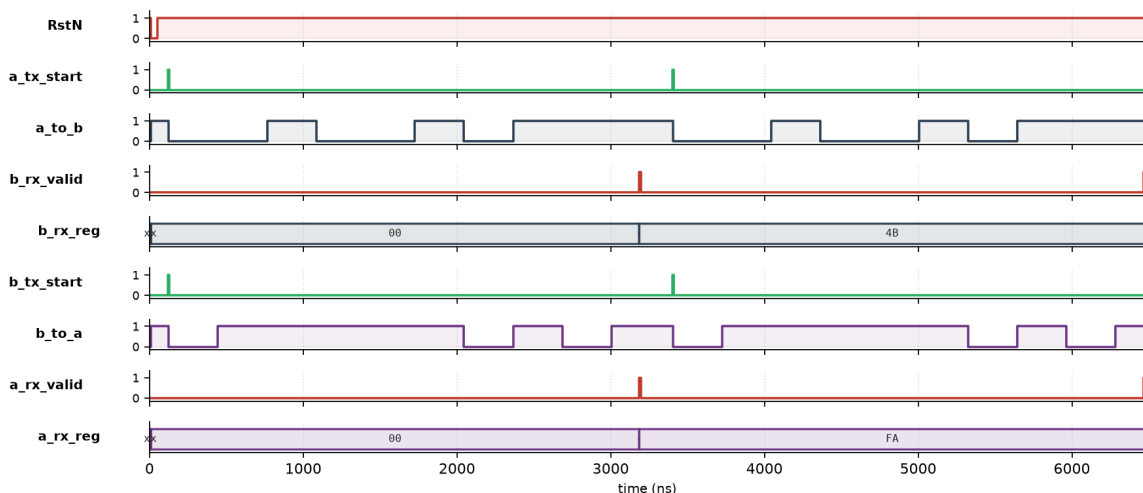
```

20 input wire a_tx_start,
21 input wire [UART_DATA_BITS-1:0] a_tx_data,
22 output wire a_tx_busy,
23 output wire a_tx_done,
24 output wire [UART_RX_REG_BITS-1:0] a_rx_reg,
25 output wire [UART_DATA_BITS-1:0] a_rx_data,
26 output wire a_rx_valid,
27 output wire a_parity_err,
28 output wire a_frame_err,
29
30 // unit B
31 input wire b_tx_start,
32 input wire [UART_DATA_BITS-1:0] b_tx_data,
33 output wire b_tx_busy,
34 output wire b_tx_done,
35 output wire [UART_RX_REG_BITS-1:0] b_rx_reg,
36 output wire [UART_DATA_BITS-1:0] b_rx_data,
37 output wire b_rx_valid,
38 output wire b_parity_err,
39 output wire b_frame_err,
40
41 // the two wires between the units
42 output wire a_to_b,
43 output wire b_to_a
44 );
45 uart #(.OS_DIVISOR(A_OS_DIVISOR), .ODD_PARITY(ODD_PARITY)) u_a (
46     .Clk(Clk), .RstN(RstN),
47     .tx_start(a_tx_start), .tx_data(a_tx_data),
48     .txd(a_to_b), .tx_busy(a_tx_busy), .tx_done(a_tx_done),
49     .rxd(b_to_a),
50     .rx_reg(a_rx_reg), .rx_data(a_rx_data), .rx_valid(a_rx_valid),
51     .parity_err(a_parity_err), .frame_err(a_frame_err),
52     .rx_busy(), .os_tick()
53 );
54
55 uart #(.OS_DIVISOR(B_OS_DIVISOR), .ODD_PARITY(ODD_PARITY)) u_b (
56     .Clk(Clk), .RstN(RstN),
57     .tx_start(b_tx_start), .tx_data(b_tx_data),
58     .txd(b_to_a), .tx_busy(b_tx_busy), .tx_done(b_tx_done),
59     .rxd(a_to_b),
60     .rx_reg(b_rx_reg), .rx_data(b_rx_data), .rx_valid(b_rx_valid),
61     .parity_err(b_parity_err), .frame_err(b_frame_err),
62     .rx_busy(), .os_tick()
63 );
64 endmodule

```

شکل موج

Two UART units exchanging characters: A sends 'K' (0x4B) while B sends 'z' (0x7A)



در این شکل موج، واحد A کاراکتر 'K' (کد 0x4B) و واحد B در همان لحظه کاراکتر 'z' (کد 0x7A) را می‌فرستد. دو سیم a_to_b و b_to_a هم‌زمان مشغول‌اند، یعنی لینک واقعا دوطرفه‌ی هم‌زمان است.

مقادیر ذخیره‌شده در ثبات‌ها نشان می‌دهد ترتیب بیت‌ها درست است:

کاراکتر	کد ASCII	توازن زوج	محتوای ثبات ۸ بیتی
'K'	1001011 = 0x4B	۰	0x4B = 0_1001011
'z'	1111010 = 0x7A	۱	0xFA = 1_1111010

توجه کنید که برای 'K' تعداد یک‌ها زوج است پس بیت توازن صفر می‌شود و محتوای ثبات با خود کد ASCII یکی درمی‌آید؛ برای 'z' تعداد یک‌ها فرد است، بیت توازن یک می‌شود و در پرارزش‌ترین جایگاه ثبات می‌نشیند. در هر دو حالت هفت بیت پایین ثبات بدون هیچ پردازش اضافه همان کد ASCII است — همان چیزی که در بخش ترتیب بیت‌ها درباره‌اش صحبت شد.

تست‌بنچ‌ها

سه تست‌بنچ نوشته شده و هر سه عمداً از یک اصل پیروی می‌کنند: فرستنده با گیرنده تست نمی‌شود.

دلیلش تجربه‌ی آزمایش قبلی است. اگر فریم را با `uart_tx` بسازیم و با `uart_rx` بخوانیم، هر برداشت اشتباهی از صورت مسئله — مثلاً جای عوضی بیت توازن — در هر دو ماژول تکرار می‌شود و تست با خیال راحت سبز می‌ماند. بنابراین:

تست‌بنچ	روش
<code>tb_uart_tx</code>	الگوی ۱۰ بیتی مورد انتظار را از روی صورت آزمایش می‌سازد و خط را وسط هر بیت نمونه‌برداری می‌کند
<code>tb_uart_rx</code>	فریم‌ها را بیت‌به‌بیت از داخل خود تست‌بنچ روی خط می‌راند
<code>tb_uart_link</code>	تنها جایی که دو ماژول با هم کار می‌کنند؛ اینجا هدف بررسی مبادله است نه قالب فریم

تست فرستنده هر ۱۲۸ کد ASCII را با هر دو قرارداد توازن زوج و فرد می‌فرستد و هر ۱۰ بیت هر فریم را جداگانه بررسی می‌کند. علاوه بر آن: بی‌کار بودن خط روی مقدار یک، درست بودن `tx_busy` در طول فریم، پالس `tx_done` در انتها، و نادیده گرفته شدن درخواست ارسال در میانه‌ی فریم.

```

1 `timescale 1ns/1ps
2 `include "uart_pkg.vh"
3
4 // Bit-exact check of the transmitted frame.
5 //
6 // This testbench deliberately does NOT use uart_rx. It rebuilds the expected
7 // 10-bit pattern from the lab sheet by hand and samples the line in the middle
8 // of every bit, so a transmitter and a receiver that share the same wrong idea
9 // about the frame layout cannot pass it together.
10 module tb_uart_tx;
11     localparam integer OSD = 3; // clocks per oversampling tick
12     localparam integer OS = `UART_OVERSAMPLE; // ticks per bit
13
14     reg Clk = 1'b0;
15     reg RstN = 1'b1;
16     reg tx_start = 1'b0;
17     reg [`UART_DATA_BITS-1:0] tx_data = 7'd0;
18
19     wire os_tick;
20     wire txd_e, busy_e, done_e;
21     wire txd_o, busy_o, done_o;
22
23     baud_gen #(.DIVISOR(OSD)) u_baud (.Clk(Clk), .RstN(RstN), .tick(os_tick));
24
25     uart_tx #(.ODD_PARITY(1'b0)) dut_even (
26         .Clk(Clk), .RstN(RstN), .os_tick(os_tick),
27         .tx_start(tx_start), .tx_data(tx_data),
28         .txd(txd_o), .tx_busy(busy_o), .tx_done(done_e)

```



```

29 );
30
31 uart_tx #(.ODD_PARITY(1'b1)) dut_odd (
32     .Clk(Clk), .RstN(RstN), .os_tick(os_tick),
33     .tx_start(tx_start), .tx_data(tx_data),
34     .txd(txd_o), .tx_busy(busy_o), .tx_done(done_o)
35 );
36
37 always #5 Clk = ~Clk;
38
39 // Free-running count of oversampling ticks; every timing decision below is
40 // expressed in ticks so it does not depend on OSD.
41 integer ntick = 0;
42 always @(posedge Clk) if (os_tick) ntick = ntick + 1;
43
44 integer pass = 0, fail = 0;
45 integer c, i, t0;
46 reg [9:0] exp;
47 reg      got_done_e, got_done_o;
48
49 always @(posedge Clk) begin
50     if (done_e) got_done_e <= 1'b1;
51     if (done_o) got_done_o <= 1'b1;
52 end
53
54 task automatic chk;
55     input [255:0] name;
56     input        ok;
57     begin
58         if (ok) pass = pass + 1;
59         else begin
60             fail = fail + 1;
61             $display(" FAIL @%0t: %0s", $time, name);
62         end
63     end
64 endtask
65
66 task automatic wait_tick;
67     input integer target;
68     begin
69         while (ntick < target) @(posedge Clk);
70         @(negedge Clk);
71     end
72 endtask
73
74 // Expected line pattern: START(0) PARITY D6..D0 STOP(1)
75 function [9:0] frame;
76     input [`UART_DATA_BITS-1:0] d;
77     input                        odd;
78     begin
79         frame = {1'b0, odd ? ~(^d) : (^d), d[6], d[5], d[4], d[3], d[2], d[1], d[0], 1'b1};
80     end
81 endfunction
82
83 initial begin
84     got_done_e = 1'b0;
85     got_done_o = 1'b0;
86
87     @(negedge Clk); RstN = 1'b0;
88     repeat (4) @(negedge Clk); RstN = 1'b1;
89     repeat (2) @(negedge Clk);
90
91     chk("line idles high (even)", txd_e === 1'b1);
92     chk("line idles high (odd)",  txd_o === 1'b1);
93     chk("not busy after reset",   busy_e === 1'b0 && busy_o === 1'b0);
94
95     // ---- every 7-bit ASCII code, both parity conventions ----
96     for (c = 0; c < 128; c = c + 1) begin
97         @(negedge Clk); tx_data = c[6:0]; tx_start = 1'b1;
98         @(negedge Clk); tx_start = 1'b0;
99         t0 = ntick;
100
101         got_done_e = 1'b0;
102         got_done_o = 1'b0;
103
104         for (i = 0; i < `UART_FRAME_BITS; i = i + 1) begin
105             wait_tick(t0 + OS*i + OS/2); // middle of bit i
106             exp = frame(c[6:0], 1'b0);
107             chk("even-parity frame bit", txd_e === exp[9-i]);
108             exp = frame(c[6:0], 1'b1);
109             chk("odd-parity frame bit",  txd_o === exp[9-i]);
110             if (i < `UART_FRAME_BITS - 1)
111                 chk("busy during frame", busy_e === 1'b1 && busy_o === 1'b1);
112         end
113
114         wait_tick(t0 + OS*`UART_FRAME_BITS + 1);
115         chk("tx_done pulsed",          got_done_e && got_done_o);

```

```

116         chk("idle again",          busy_e === 1'b0 && busy_o === 1'b0);
117         chk("line back to idle 1", txd_e === 1'b1 && txd_o === 1'b1);
118     end
119
120     // ---- a request while busy must be ignored, not queued ----
121     @(negedge Clk); tx_data = 7'h55; tx_start = 1'b1;
122     @(negedge Clk); tx_start = 1'b0;
123     t0 = ntick;
124     wait_tick(t0 + OS + OS/2); // sitting in the parity bit
125     @(negedge Clk); tx_data = 7'h2A; tx_start = 1'b1;
126     @(negedge Clk); tx_start = 1'b0;
127     for (i = 1; i < `UART_FRAME_BITS; i = i + 1) begin
128         wait_tick(t0 + OS*i + OS/2);
129         exp = frame(7'h55, 1'b0);
130         chk("frame unaffected by mid-frame request", txd_e === exp[9-i]);
131     end
132
133     $display("tb_uart_tx: Passed: %0d, Failed: %0d", pass, fail);
134     if (fail != 0) $fatal(1);
135     $finish;
136 end
137 endmodule

```

تست گیرنده همان ۱۲۸ کد را با هر دو قرارداد توازن روی خط می‌راند و علاوه بر آن چهار وضعیت غیرعادی را می‌سازد: بیت توازن خراب، بیت خاتمه‌ی گم‌شده، یک سوزن کوتاه روی خط، و فریم‌های پشت سر هم بدون هیچ فاصله‌ی بی‌کاری.

نکته‌ی ظریفی که این تست روشن می‌کند: پس از خطای توازن، داده همچنان در ثبات می‌نشیند و فقط پرچم خطا بالا می‌رود. تصمیم درباره‌ی دور انداختن یا نینداختن کاراکتر خراب کار لایه‌ی بالاتر است، نه کار UART.

```

1  `timescale 1ns/1ps
2  `include "uart_pkg.vh"
3
4  // Receiver test. Frames are driven onto the line bit by bit from this file,
5  // again without using uart_tx, so the receiver is checked against the lab
6  // sheet rather than against its own transmitter.
7  //
8  // Covered: every ASCII code, both parity conventions, a corrupted parity bit,
9  // a missing stop bit (framing error + resynchronisation), a glitch that must
10 // not be mistaken for a start bit, and back-to-back frames with no idle gap.
11 module tb_uart_rx;
12     localparam integer OSD = 3;
13     localparam integer OS = `UART_OVERSAMPLE;
14
15     reg Clk = 1'b0, RstN = 1'b1, line = 1'b1;
16     wire os_tick;
17
18     wire [`UART_RX_REG_BITS-1:0] reg_e, reg_o;
19     wire [`UART_DATA_BITS-1:0] dat_e, dat_o;
20     wire vld_e, per_e, fer_e, bsy_e;
21     wire vld_o, per_o, fer_o, bsy_o;
22
23     baud_gen #(.DIVISOR(OSD)) u_baud (.Clk(Clk), .RstN(RstN), .tick(os_tick));
24
25     uart_rx #(.ODD_PARITY(1'b0)) dut_even (
26         .Clk(Clk), .RstN(RstN), .os_tick(os_tick), .rxd(line),
27         .rx_reg(reg_e), .rx_data(dat_e), .rx_valid(vld_e),
28         .parity_err(per_e), .frame_err(fer_e), .rx_busy(bsy_e)
29     );
30
31     uart_rx #(.ODD_PARITY(1'b1)) dut_odd (
32         .Clk(Clk), .RstN(RstN), .os_tick(os_tick), .rxd(line),
33         .rx_reg(reg_o), .rx_data(dat_o), .rx_valid(vld_o),
34         .parity_err(per_o), .frame_err(fer_o), .rx_busy(bsy_o)
35     );
36
37     always #5 Clk = ~Clk;
38
39     integer ntick = 0;
40     always @(posedge Clk) if (os_tick) ntick = ntick + 1;
41
42     integer pass = 0, fail = 0;
43     integer c, i;
44
45     // Latched copies of the one-cycle rx_valid pulse and its flags.
46     reg seen_e, seen_o;
47     reg [`UART_RX_REG_BITS-1:0] cap_e, cap_o;
48     reg cpe_e, cfe_e, cpe_o, cfe_o;
49
50     always @(posedge Clk) begin
51         if (vld_e) begin seen_e <= 1'b1; cap_e <= reg_e; cpe_e <= per_e; cfe_e <= fer_e; end
52         if (vld_o) begin seen_o <= 1'b1; cap_o <= reg_o; cpe_o <= per_o; cfe_o <= fer_o; end

```

```

53 end
54
55 task automatic chk;
56     input [255:0] name;
57     input      ok;
58     begin
59         if (ok) pass = pass + 1;
60         else begin
61             fail = fail + 1;
62             $display("  FAIL @%0t: %0s", $time, name);
63         end
64     end
65 endtask
66
67 task automatic hold_ticks;                // hold the line for n tick times
68     input integer n;
69     integer target;
70     begin
71         target = ntick + n;
72         while (ntick < target) @(posedge Clk);
73         @(negedge Clk);
74     end
75 endtask
76
77 task automatic drive_bit;
78     input b;
79     begin
80         line = b;
81         hold_ticks(0S);
82     end
83 endtask
84
85 task automatic clear_flags;
86     begin
87         seen_e = 1'b0; seen_o = 1'b0;
88         cpe_e  = 1'b0; cfe_e  = 1'b0;
89         cpe_o  = 1'b0; cfe_o  = 1'b0;
90     end
91 endtask
92
93 // START(0) PARITY D6..D0 STOP -- par/stop overridable to inject errors.
94 task automatic drive_frame;
95     input [`UART_DATA_BITS-1:0] d;
96     input                       par;
97     input                       stop;
98     integer k;
99     begin
100         drive_bit(1'b0);
101         drive_bit(par);
102         for (k = `UART_DATA_BITS-1; k >= 0; k = k - 1) drive_bit(d[k]);
103         drive_bit(stop);
104     end
105 endtask
106
107 initial begin
108     clear_flags;
109     @(negedge Clk); RstN = 1'b0;
110     repeat (4) @(negedge Clk); RstN = 1'b1;
111     hold_ticks(2);
112
113     chk("idle after reset", bsy_e === 1'b0 && bsy_o === 1'b0);
114
115     // ---- every ASCII code, correct frame for each parity convention ----
116     for (c = 0; c < 128; c = c + 1) begin
117         clear_flags;
118         drive_frame(c[6:0], ^c[6:0], 1'b1);          // even parity on the wire
119         hold_ticks(2);
120         chk("even: frame accepted", seen_e);
121         chk("even: register value", cap_e === {^c[6:0], c[6:0]});
122         chk("even: no parity error", cpe_e === 1'b0);
123         chk("even: no frame error", cfe_e === 1'b0);
124         chk("even: data output", dat_e === c[6:0]);
125         chk("odd DUT flags parity", seen_o && cpe_o === 1'b1);
126
127         clear_flags;
128         drive_frame(c[6:0], ~(^c[6:0]), 1'b1);      // odd parity on the wire
129         hold_ticks(2);
130         chk("odd: frame accepted", seen_o);
131         chk("odd: register value", cap_o === {~(^c[6:0]), c[6:0]});
132         chk("odd: no parity error", cpe_o === 1'b0);
133         chk("even DUT flags parity", seen_e && cpe_e === 1'b1);
134     end
135
136     // ---- a corrupted parity bit is reported but the data still lands ----
137     clear_flags;
138     drive_frame(7'h41, ~(^7'h41), 1'b1);
139     hold_ticks(2);

```

```

140     chk("bad parity reported",      seen_e && cpe_e == 1'b1);
141     chk("data still captured",      cap_e[UART_DATA_BITS-1:0] == 7'h41);
142
143     // ---- missing stop bit -> framing error, then resynchronise ----
144     clear_flags;
145     drive_frame(7'h55, ^7'h55, 1'b0);
146     hold_ticks(2);
147     chk("framing error reported",    seen_e && cfe_e == 1'b1);
148     line = 1'b1; hold_ticks(4);      // line returns to idle
149     clear_flags;
150     drive_frame(7'h2A, ^7'h2A, 1'b1);
151     hold_ticks(2);
152     chk("recovers after frame error", seen_e && cap_e == {^7'h2A, 7'h2A});
153     chk("flags cleared on good frame", cfe_e == 1'b0 && cpe_e == 1'b0);
154
155     // ---- a short low glitch must not start a reception ----
156     clear_flags;
157     line = 1'b0; hold_ticks(OS/4);
158     line = 1'b1; hold_ticks(2*OS);
159     chk("glitch rejected", !seen_e && !seen_o && bsy_e == 1'b0);
160
161     // ---- back-to-back frames, no idle time between them ----
162     clear_flags;
163     drive_frame(7'h7F, ^7'h7F, 1'b1);
164     chk("first of a burst", seen_e && cap_e == {^7'h7F, 7'h7F});
165     clear_flags;
166     drive_frame(7'h00, ^7'h00, 1'b1);
167     chk("second of a burst", seen_e && cap_e == {^7'h00, 7'h00});
168     clear_flags;
169     drive_frame(7'h3C, ^7'h3C, 1'b1);
170     hold_ticks(2);
171     chk("third of a burst", seen_e && cap_e == {^7'h3C, 7'h3C});
172
173     $display("tb_uart_rx: Passed: %0d, Failed: %0d", pass, fail);
174     if (fail != 0) $fatal(1);
175     $finish;
176 end
177 endmodule

```

تست لینک دو سناریو دارد. در اولی دو واحد کاملاً هم‌نرخ‌اند و هر ۱۲۸ کاراکتر هم‌زمان در هر دو جهت رد و بدل می‌شود (واحد A کاراکتر و واحد B مکمل بیتی همان کاراکتر را می‌فرستد تا دو جهت هرگز الگوی یکسان حمل نکنند).

در دومی نسبت تقسیم دو واحد ۲۰ و ۲۱ گرفته شده است، یعنی اختلاف نرخ بیت ۵٪. این عدد اتفاقی نیست؛ درست زیر حد نظری قرار دارد. آخرین بیتی که گیرنده نمونه می‌گیرد بیت خاتمه است که ۹/۵ بیت پس از نقطه‌ی هم‌گامی قرار دارد و نمونه تا وقتی درست است که رانش انباشته از نصف بیت بیشتر نشود:

$$\text{حد اختلاف نرخ} = \frac{0.5}{9.5} \approx 5.3\%$$

این حد به صورت تجربی هم اندازه‌گیری شد: با جاروب چند لینک با اختلاف‌های مختلف، تا ۵٪ همه‌ی کاراکترها سالم می‌رسند و از ۶٪ به بعد هیچ‌کدام نمی‌رسند. یعنی مدار واقعا به همان حدی می‌رسد که تئوری می‌گوید، نه کمتر.

```

1  `timescale 1ns/1ps
2  `include "uart_pkg.vh"
3
4  // The demonstration the lab sheet asks for: two UART units wired to each
5  // other. Two independent links are instantiated.
6  //
7  //   link_ok      both units use the same divisor -- a perfectly matched pair.
8  //               Every one of the 128 ASCII codes is exchanged in BOTH
9  //               directions at the same time, which also proves full duplex.
10 //
11 //   link_skew    the two units' divisors differ by ~5%, modelling two boards
12 //               whose crystals disagree. That is close to the theoretical
13 //               limit: the last sample of a frame is taken 9.5 bit times after
14 //               the receiver synchronises, so the drift may not exceed half a
15 //               bit, i.e. 0.5 / 9.5 = 5.3%. Measured on this design, 5% still
16 //               works and 6% does not, so this link runs at 5% -- just inside
17 //               the limit. A shorter sweep is used here only
18 //               because the larger divisor makes it slow to simulate.
19 module tb_uart_link;
20     localparam integer OS = `UART_OVERSAMPLE;
21
22     reg Clk = 1'b0, RstN = 1'b1;
23     always #5 Clk = ~Clk;

```

```

24
25 reg a_start = 1'b0, b_start = 1'b0;
26 reg [`UART_DATA_BITS-1:0] a_data = 7'd0, b_data = 7'd0;
27 reg s_astart = 1'b0, s_bstart = 1'b0;
28 reg [`UART_DATA_BITS-1:0] s_adata = 7'd0, s_bdata = 7'd0;
29
30 // --- matched link ---
31 wire [`UART_RX_REG_BITS-1:0] ok_arx, ok_brx;
32 wire ok_avld, ok_bvld, ok_ape, ok_afe, ok_bpe, ok_bfe;
33 wire ok_abusy, ok_bbussy, ok_adone, ok_bdone, ok_ab, ok_ba;
34
35 uart_link #(.A_OS_DIVISOR(3), .B_OS_DIVISOR(3)) link_ok (
36     .Clk(Clk), .RstN(RstN),
37     .a_tx_start(a_start), .a_tx_data(a_data),
38     .a_tx_busy(ok_abusy), .a_tx_done(ok_adone),
39     .a_rx_reg(ok_arx), .a_rx_data(), .a_rx_valid(ok_avld),
40     .a_parity_err(ok_ape), .a_frame_err(ok_afe),
41     .b_tx_start(b_start), .b_tx_data(b_data),
42     .b_tx_busy(ok_bbussy), .b_tx_done(ok_bdone),
43     .b_rx_reg(ok_brx), .b_rx_data(), .b_rx_valid(ok_bvld),
44     .b_parity_err(ok_bpe), .b_frame_err(ok_bfe),
45     .a_to_b(ok_ab), .b_to_a(ok_ba)
46 );
47
48 // --- mismatched link: 20 vs 21 clocks per oversampling tick (5.0%) ---
49 wire [`UART_RX_REG_BITS-1:0] sk_arx, sk_brx;
50 wire sk_avld, sk_bvld, sk_ape, sk_afe, sk_bpe, sk_bfe, sk_abusy, sk_bbussy;
51
52 uart_link #(.A_OS_DIVISOR(20), .B_OS_DIVISOR(21)) link_skew (
53     .Clk(Clk), .RstN(RstN),
54     .a_tx_start(s_astart), .a_tx_data(s_adata),
55     .a_tx_busy(sk_abusy), .a_tx_done(),
56     .a_rx_reg(sk_arx), .a_rx_data(), .a_rx_valid(sk_avld),
57     .a_parity_err(sk_ape), .a_frame_err(sk_afe),
58     .b_tx_start(s_bstart), .b_tx_data(s_bdata),
59     .b_tx_busy(sk_bbussy), .b_tx_done(),
60     .b_rx_reg(sk_brx), .b_rx_data(), .b_rx_valid(sk_bvld),
61     .b_parity_err(sk_bpe), .b_frame_err(sk_bfe),
62     .a_to_b(), .b_to_a()
63 );
64
65 integer pass = 0, fail = 0;
66 integer c;
67
68 // Latch each unit's one-cycle rx_valid pulse together with its payload.
69 reg okA_seen, okB_seen, skA_seen, skB_seen;
70 reg [`UART_RX_REG_BITS-1:0] okA_reg, okB_reg, skA_reg, skB_reg;
71 reg okA_err, okB_err, skA_err, skB_err;
72
73 always @(posedge Clk) begin
74     if (ok_avld) begin okA_seen <= 1'b1; okA_reg <= ok_arx; okA_err <= ok_ape | ok_afe; end
75     if (ok_bvld) begin okB_seen <= 1'b1; okB_reg <= ok_brx; okB_err <= ok_bpe | ok_bfe; end
76     if (sk_avld) begin skA_seen <= 1'b1; skA_reg <= sk_arx; skA_err <= sk_ape | sk_afe; end
77     if (sk_bvld) begin skB_seen <= 1'b1; skB_reg <= sk_brx; skB_err <= sk_bpe | sk_bfe; end
78 end
79
80 task automatic chk;
81     input [255:0] name;
82     input ok;
83     begin
84         if (ok) pass = pass + 1;
85         else begin
86             fail = fail + 1;
87             $display(" FAIL %0t: %0s", $time, name);
88         end
89     end
90 endtask
91
92 // Fire both transmitters of the matched link on the same clock edge.
93 task automatic exchange;
94     input [`UART_DATA_BITS-1:0] da;
95     input [`UART_DATA_BITS-1:0] db;
96     begin
97         // rx_valid fires in the middle of the stop bit, so the far
98         // transmitter is still driving the line for another half bit.
99         // Wait for it to release before loading the next character.
100         wait (!ok_abusy && !ok_bbussy);
101         okA_seen = 1'b0; okB_seen = 1'b0; okA_err = 1'b0; okB_err = 1'b0;
102         @(negedge Clk); a_data = da; b_data = db; a_start = 1'b1; b_start = 1'b1;
103         @(negedge Clk); a_start = 1'b0; b_start = 1'b0;
104         wait (okA_seen && okB_seen);
105         @(negedge Clk);
106     end
107 endtask
108
109 task automatic exchange_skew;
110     input [`UART_DATA_BITS-1:0] da;

```

```

111     input [`UART_DATA_BITS-1:0] db;
112     begin
113         wait (!sk_abusy && !sk_bbusy);
114         skA_seen = 1'b0; skB_seen = 1'b0; skA_err = 1'b0; skB_err = 1'b0;
115         @(negedge Clk); s_data = da; s_bdata = db; s_astart = 1'b1; s_bstart = 1'b1;
116         @(negedge Clk); s_astart = 1'b0; s_bstart = 1'b0;
117         wait (skA_seen && skB_seen);
118         @(negedge Clk);
119     end
120 endtask
121
122 initial begin
123     okA_seen = 1'b0; okB_seen = 1'b0; skA_seen = 1'b0; skB_seen = 1'b0;
124     okA_err = 1'b0; okB_err = 1'b0; skA_err = 1'b0; skB_err = 1'b0;
125
126     @(negedge Clk); RstN = 1'b0;
127     repeat (4) @(negedge Clk); RstN = 1'b1;
128     repeat (4) @(negedge Clk);
129
130     chk("both lines idle high", ok_ab == 1'b1 && ok_ba == 1'b1);
131
132     // --- full-duplex sweep on the matched link ---
133     // A sends c while B sends its bit-complement, so the two directions
134     // never carry the same pattern at the same time.
135     for (c = 0; c < 128; c = c + 1) begin
136         exchange(c[6:0], ~c[6:0]);
137         chk("B received A's character", okB_reg == {~c[6:0], c[6:0]});
138         chk("A received B's character", okA_reg == {~(c[6:0]), ~c[6:0]});
139         chk("no errors on either end", !okA_err && !okB_err);
140     end
141
142     // --- a burst: the next character is loaded on the very clock the
143     // previous frame ends, so the line never returns to idle in between ---
144     wait (!ok_abusy);
145     okB_seen = 1'b0;
146     @(negedge Clk); a_data = 7'h48; a_start = 1'b1; // 'H'
147     @(negedge Clk); a_start = 1'b0;
148     @(posedge ok_adone);
149     @(negedge Clk); a_data = 7'h69; a_start = 1'b1; // 'i'
150     @(negedge Clk); a_start = 1'b0;
151     wait (okB_seen); chk("burst 'H'", okB_reg[`UART_DATA_BITS-1:0] == 7'h48);
152     okB_seen = 1'b0;
153     wait (okB_seen); chk("burst 'i'", okB_reg[`UART_DATA_BITS-1:0] == 7'h69);
154     wait (!ok_abusy && !ok_bbusy);
155
156     chk("both units idle at the end", !ok_abusy && !ok_bbusy);
157
158     // --- mismatched baud rates ---
159     for (c = 0; c < 16; c = c + 1) begin
160         exchange_skew({c[3:0], 3'b101}, {3'b010, c[3:0]});
161         chk("skewed link: B received A", skB_reg == {~{c[3:0], 3'b101}, c[3:0], 3'b101});
162         chk("skewed link: A received B", skA_reg == {~{3'b010, c[3:0]}, 3'b010, c[3:0]});
163         chk("skewed link: no errors", !skA_err && !skB_err);
164     end
165
166     $display("tb_uart_link: Passed: %0d, Failed: %0d", pass, fail);
167     if (fail != 0) $fatal(1);
168     $finish;
169 end
170 endmodule

```

اندازه‌گیری نقطه‌ی نمونه‌برداری

ادعای «نمونه‌برداری از وسط بیت» چیزی است که به راحتی می‌شود نوشت و اشتباه پیاده‌سازی کرد، چون یک نمونه‌ی کمی دور از وسط هم در لینک هم‌نرخ کاملاً درست کار می‌کند و هیچ تستی را رد نمی‌کند. تنها اثرش این است که حاشیه‌ی تحمل اختلاف نرخ بی‌سروصدا کم می‌شود.

بنابراین نقطه‌ی نمونه‌برداری مستقیماً اندازه‌گیری شد: شمارش پالس‌های `os_tick` از لحظه‌ی پایین آمدن خط تا لحظه‌ای که هر بیت در ثبات می‌نشیند. نقاط ایده‌آل ۸، ۲۴، ۴۰، ... هستند.

اولین اندازه‌گیری اعداد ۲۶، ۴۲، ۵۸، ... را داد؛ یعنی هر نمونه دو تیک دیرتر از وسط بیت گرفته می‌شد. علتش یک خطای واحد در شمارنده‌ی نیم‌بیت بود:

```

if (os == OS/2) // wrong: os starts at 0 and is tested BEFORE
                // incrementing, so this waits OS/2 + 1 ticks
if (os == OS/2 - 1) // right: counting 0..OS/2-1 is exactly OS/2 ticks

```

پس از اصلاح، اعداد به ۲۵، ۴۱، ۵۷، ... رسیدند. یک تیک باقی‌مانده اجتناب‌ناپذیر است: شمارنده‌ی داخل بیت فقط روی مرز پالس‌های نمونه‌برداری می‌تواند شروع شود، پس تا $\frac{1}{16}$ بیت خطای اولیه همیشه وجود دارد — همان چیزی که در تحلیل حاشیه‌ی خطا هم لحاظ شده است.

اثر این یک تیک کم نیست: پیش از اصلاح، لینک با اختلاف نرخ ۵٪ کاراکتر گم می‌کرد و اکنون نمی‌کند.

نتایج تست

```
iverilog: Icarus Verilog version 12.0 (stable) ()

=== tb_uart_tx ===
tb_uart_tx: Passed: 4108, Failed: 0
PASS: tb_uart_tx

=== tb_uart_rx ===
tb_uart_rx: Passed: 1290, Failed: 0
PASS: tb_uart_rx

=== tb_uart_link ===
tb_uart_link: Passed: 436, Failed: 0
PASS: tb_uart_link

summary: 3/3 passed, 0 failed
```

جمع‌بندی

UART در نگاه اول یک ثبات لغزشی ساده به نظر می‌رسد، ولی تمام دشواری آن در سمت گیرنده و در نبود کلاک مشترک است. سه تصمیمی که این طراحی را کار می‌اندازند عبارت‌اند از: نمونه‌برداری ۱۶ برابر برای تشخیص دقیق لبه‌ی شروع، تایید مجدد بیت شروع پس از نیم بیت برای رد کردن نویز، و نمونه‌برداری از وسط بیت برای گرفتن بیشترین حاشیه نسبت به اختلاف نرخ دو طرف.

تصمیم طراحی دیگری که ارزش تکرار دارد، MSB اول فرستادن داده است. این انتخاب به‌خودی‌خود «درست‌تر» نیست؛ درست است چون خواسته‌ی صورت آزمایش — ذخیره‌ی توازن و داده در یک ثبات ۸ بیتی — را بدون هیچ منطق اضافه‌ای برآورده می‌کند. قالب فریم و ساختار داخلی گیرنده دو روی یک سکه‌اند و انتخاب هماهنگی آن‌ها یک معکوس‌کننده‌ی بیتی کامل را از مدار حذف می‌کند.